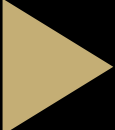


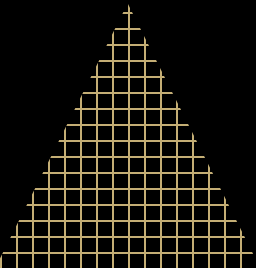


CS-218 Data Structures and Algorithms



LECTURE 23

BY UROOJ AINUDDIN

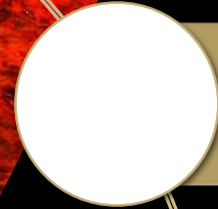




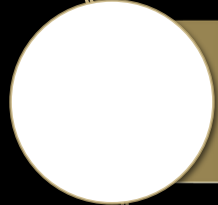
Trees

BOOK 1 CHAPTER 13





We were introduced to breadth-first tree traversal algorithm.



We were introduced to the heap.



We learned how to add to, build and delete from a heap.



We examined all codes for time and space complexities.

In the last lecture...

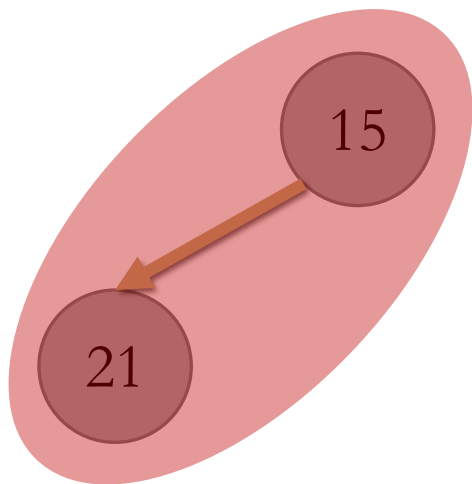


Heapsort

Given a list of n elements:

1. Build a heap, stored in a Python list `mylist`. Length of the heap is n (from 0 to $n - 1$).
2. For i from 0 to $n - 2$, step 1:
 - a) Swap elements 0 and $n - 1 - i$. Length of the heap is $n - 1 - i$ now (from 0 to $n - 1 - i - 2$).
 - b) Sink the root in the shortened heap.
3. Return `mylist` to the user. It now contains the elements sorted in ascending order.



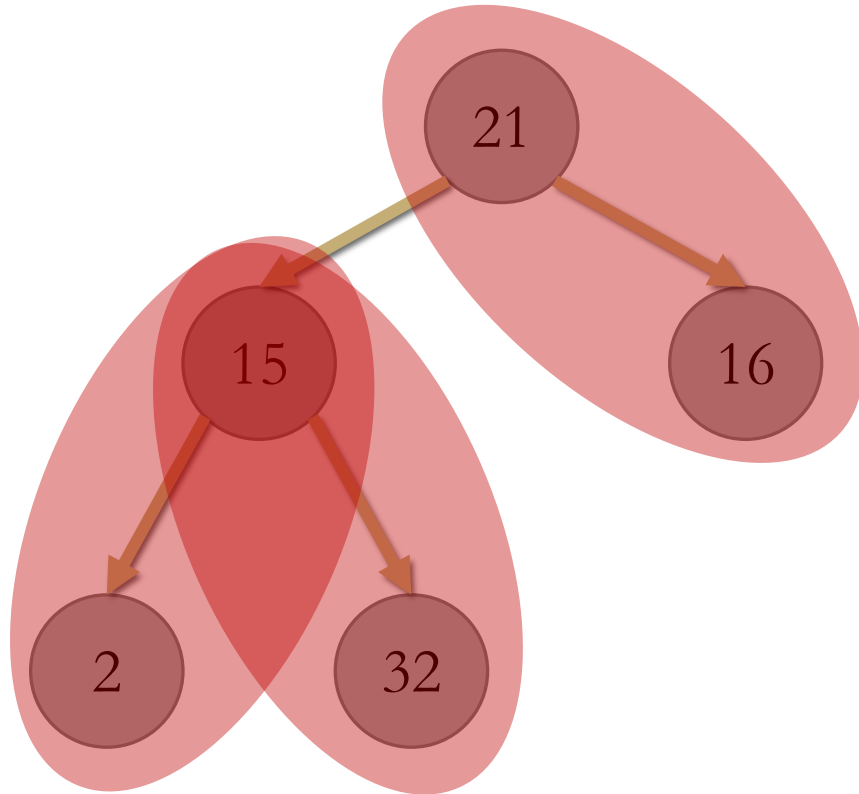


15	21
0	1

Heapsort

Sort the list
[15,21,16,2,32].



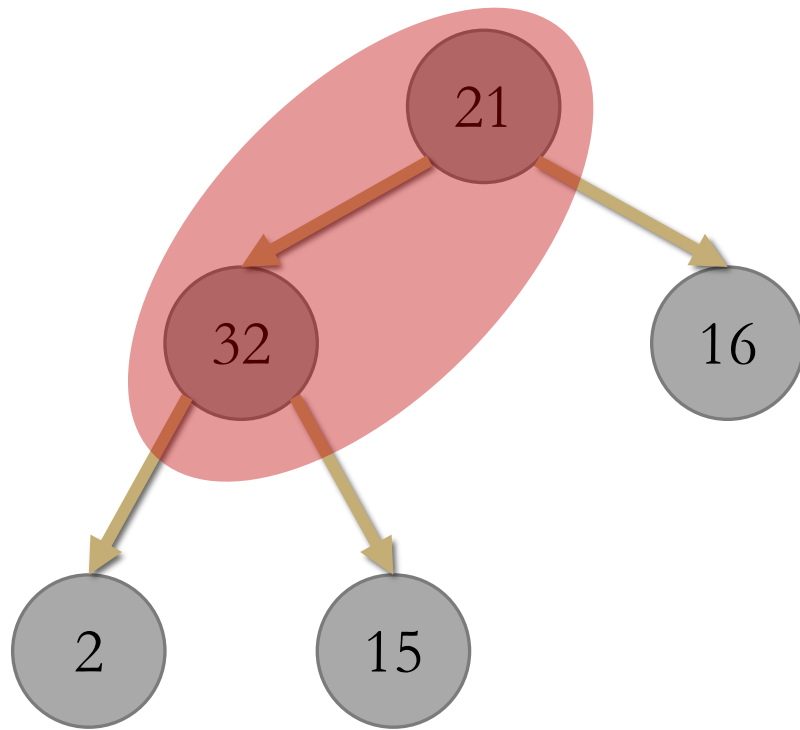


21	15	16	2	32
0	1	2	3	4

Heapsort

Sort the list
[15,21,16,2,32].





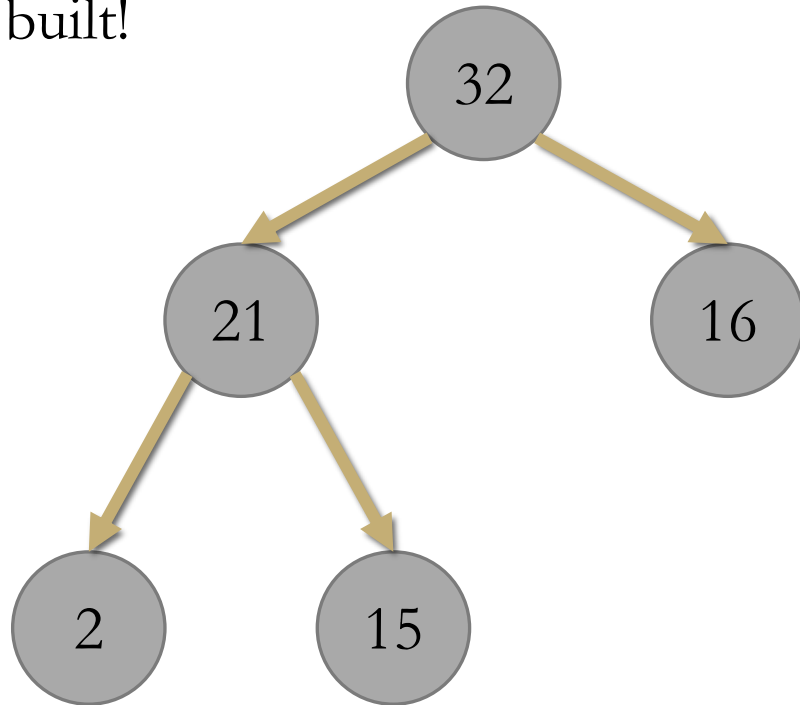
21	32	16	2	15
0	1	2	3	4

Heapsort

Sort the list
[15,21,16,2,32].



Heap built!



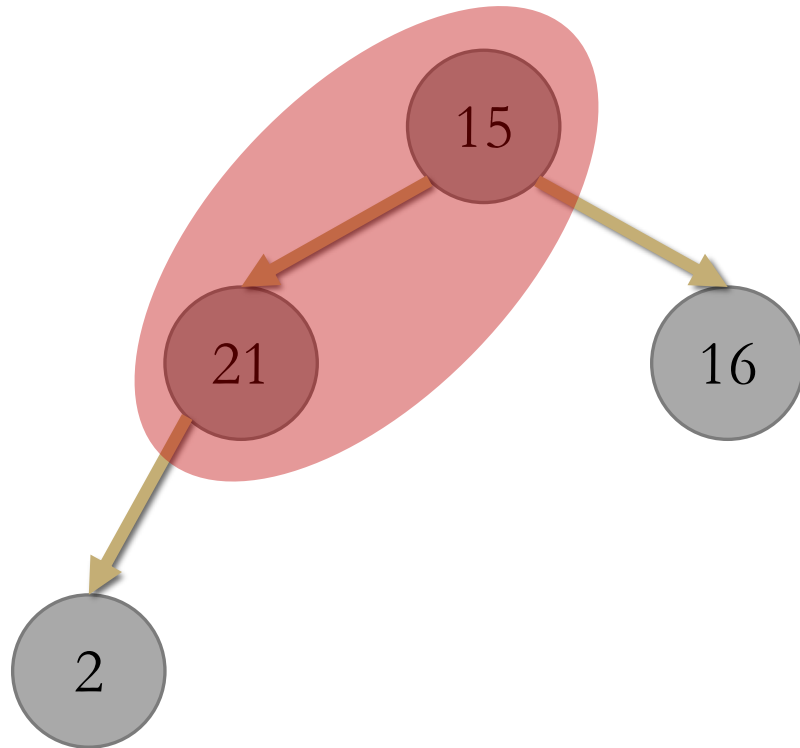
32	21	16	2	15
0	1	2	3	4

Heapsort

Sort the list
[15,21,16,2,32].



$i = 0$



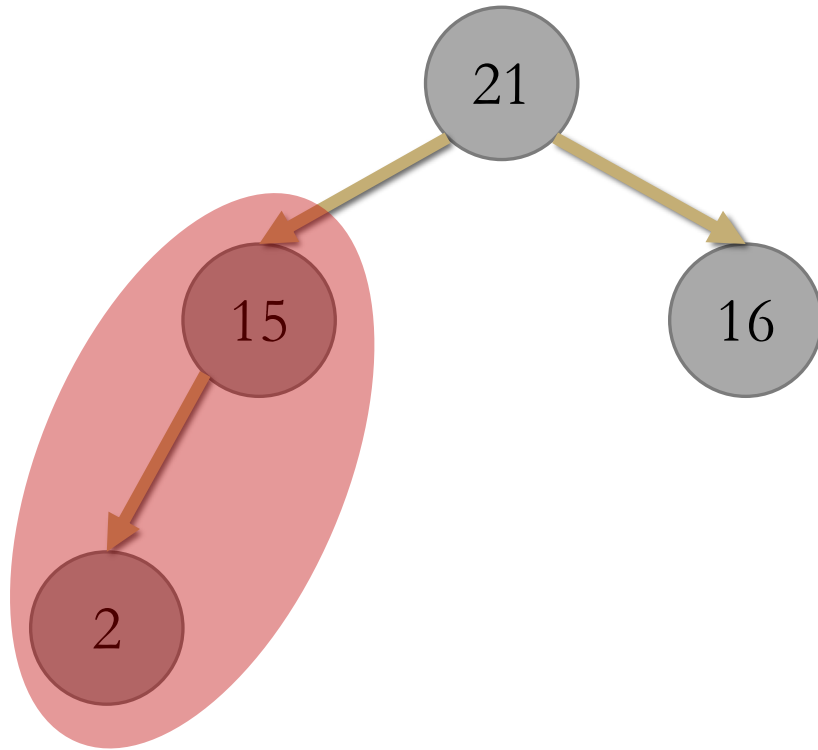
15	21	16	2	32
0	1	2	3	4

Heapsort

Sort the list
[15,21,16,2,32].



$i = 0$



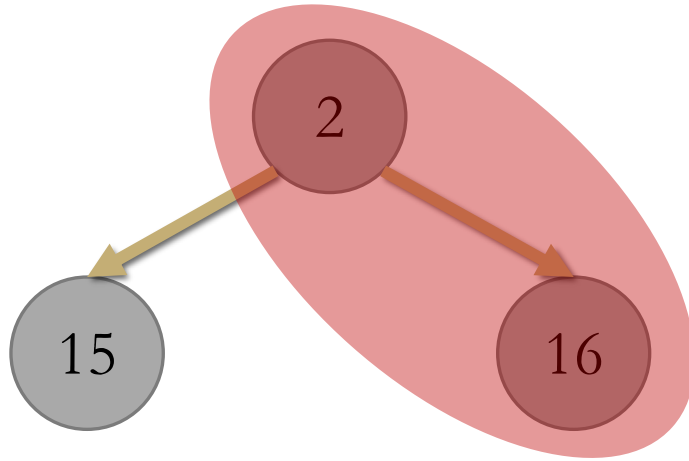
21	15	16	2	32
0	1	2	3	4

Heapsort

Sort the list
[15,21,16,2,32].



$i = 1$



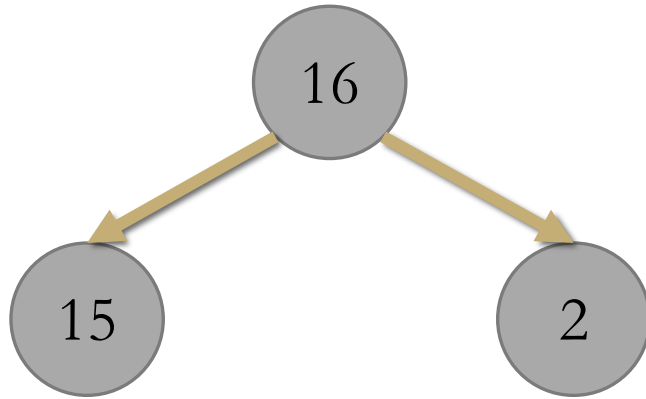
2	15	16	21	32
0	1	2	3	4

Heapsort

Sort the list
[15,21,16,2,32].



$i = 2$



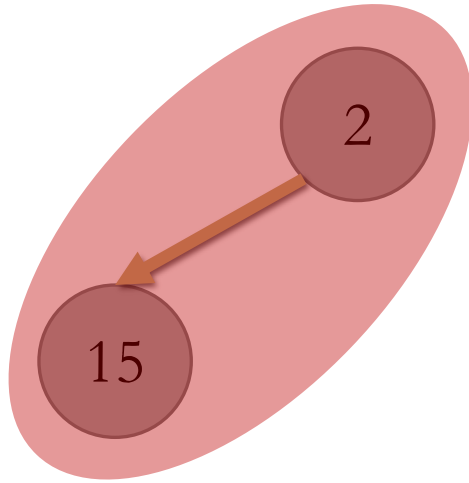
16	15	2	21	32
0	1	2	3	4

Heapsort

Sort the list
[15,21,16,2,32].



$i = 2$



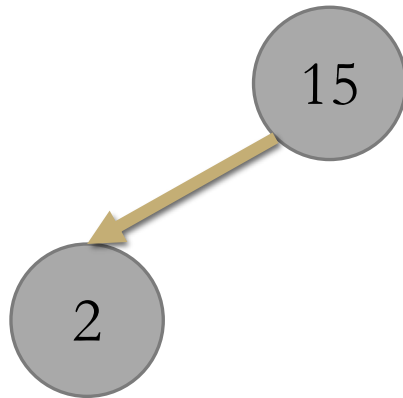
2	15	16	21	32
0	1	2	3	4

Heapsort

Sort the list
[15,21,16,2,32].



$i = 2$



15	2	16	21	32
0	1	2	3	4

Heapsort

Sort the list
[15,21,16,2,32].



$i = 3$



List sorted!

2	15	16	21	32
0	1	2	3	4

Heapsort

Sort the list
[15,21,16,2,32].



Heapsort

- The first part of heapsort builds the heap.
- It takes $O(n \log n)$ time to build the heap.
- The second part of heapsort does $n - 1$ deletions.
- Each deletion takes $O(\log n)$ time.
- Time for the second part of heapsort is $O(n \log n)$.
- Since the two parts happen one after the other, their times are added.

Time: $O(n \log n)$



Heapsort

- Heapsort uses a Python list of n elements to hold the heap.
- It builds the heap in this list.
- It sorts the elements while they remain in this list. It is an in-place or in-situ sorting algorithm.
- It uses a temporary variable for swapping.

Space: $O(n)$



Search trees

BOOK 1 CHAPTER 14





Binary search tree (BST)

- A binary search tree (BST) is a binary tree in which for every node x , the following is true:
 - All nodes in the left subtree of x contain values less than that of node x .
 - All nodes in the right subtree of x contain values greater than that of node x .
- An inorder traversal of a binary search tree yields a traversal sequence sorted in ascending order!



Input: a Python list called `val` containing input values.

1. Set `root = BTree(val[0])`.
2. For `i` from 1 to `len(val) - 1`, step 1:
 - a) Add `val[i]` to root.
3. Return root.

Building a binary search tree

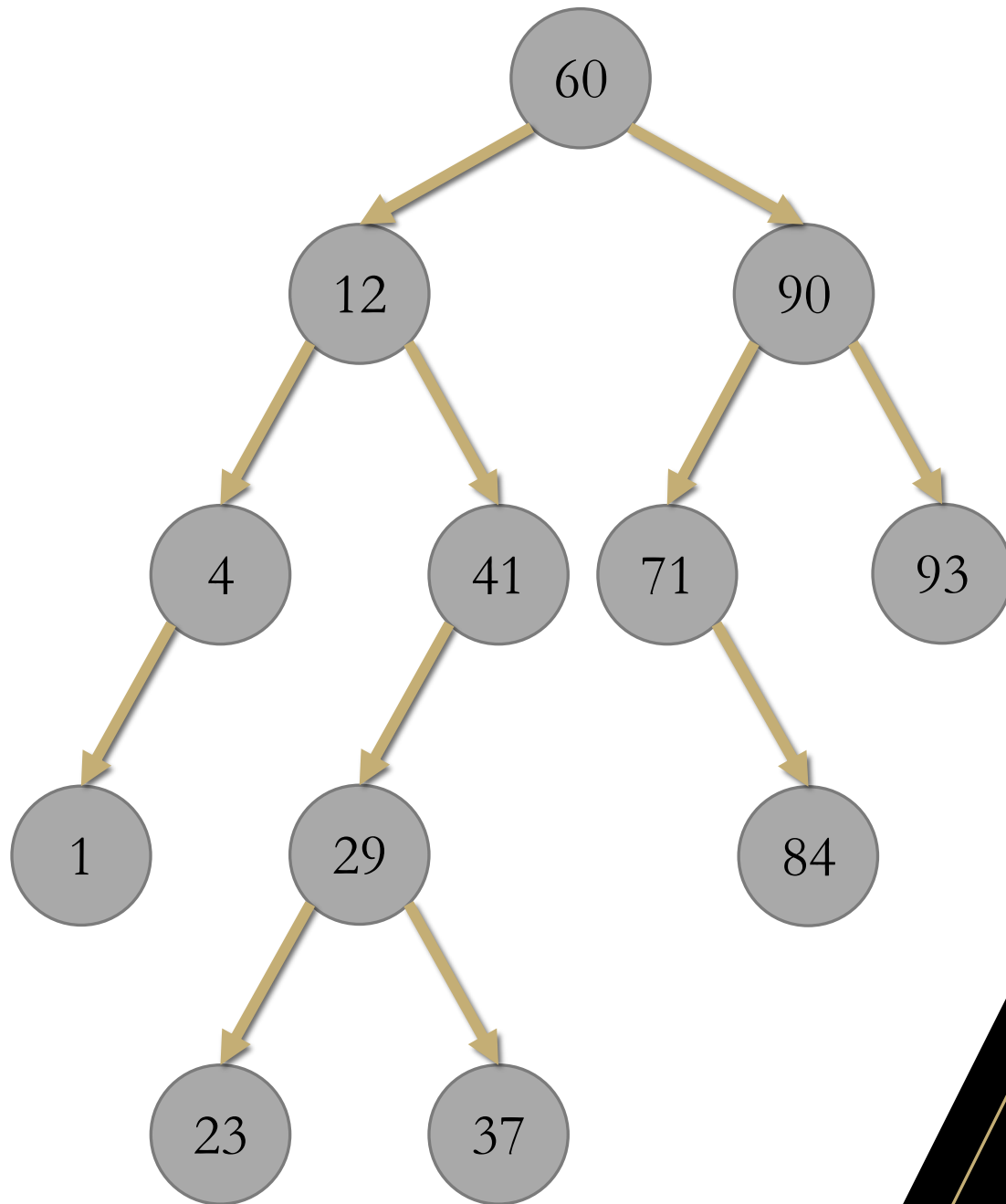


Adding to a binary search tree

Input: pointer to the root of the BST called **node**, value to insert called **item**.

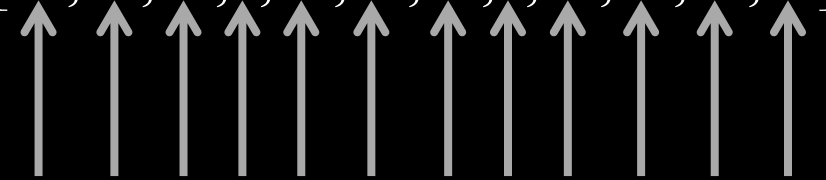
1. Set `parent = node` and `added = False`.
2. While not added:
 - a) If `item == parent.data`, `added = True`.
 - b) If `item < parent.data`:
 - i. If `parent.lc` is present, `parent = parent.lc`.
 - ii. Else add `item` as left child to `parent`, `added = True`.
 - c) Else:
 - i. If `parent.rc` is present, `parent = parent.rc`.
 - ii. Else add `item` as right child to `parent`, `added = True`.
3. Return root.

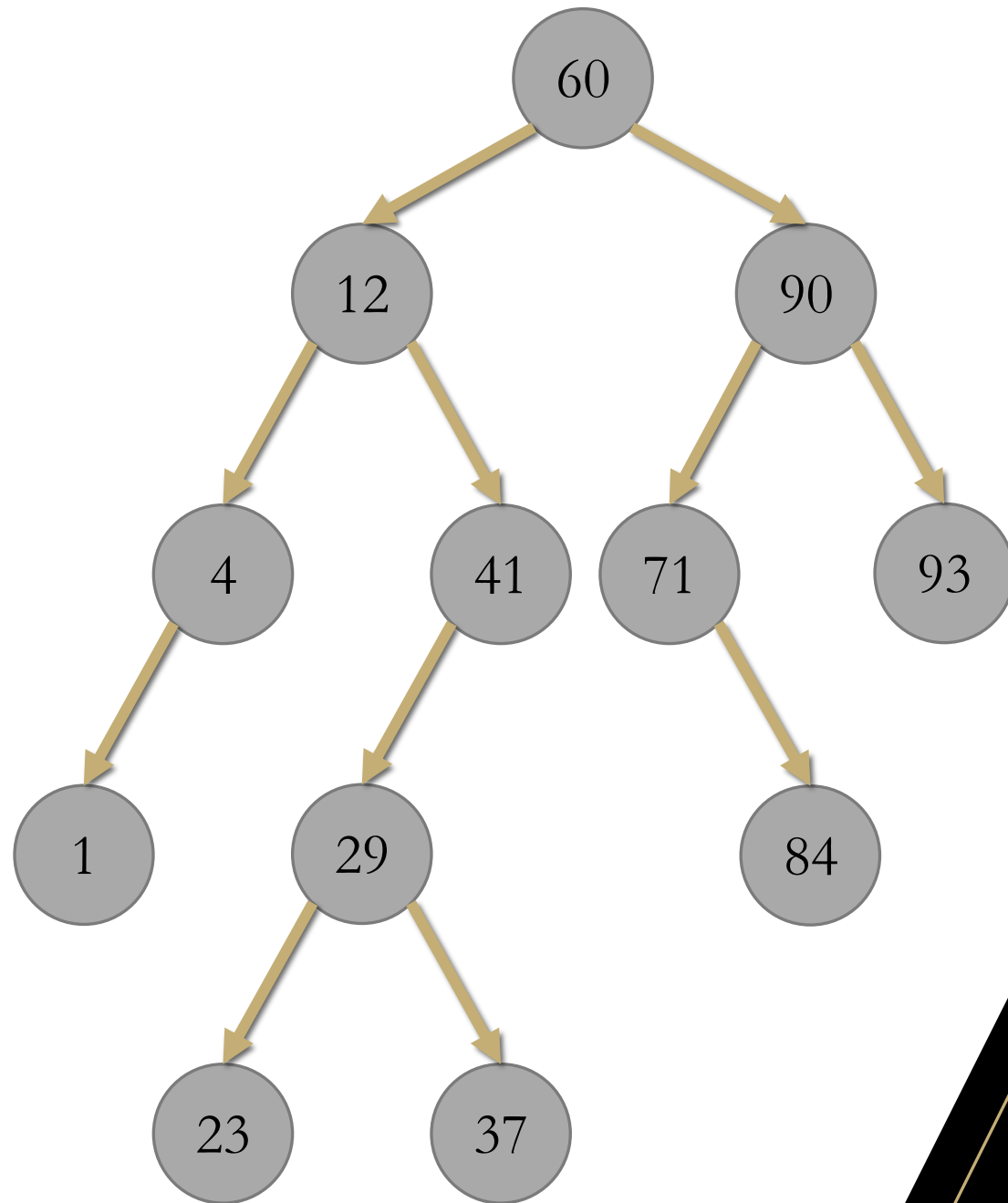




Building a BST

Build a BST from
[60,12,90,4,41,71,93,1,29,84,23,37].



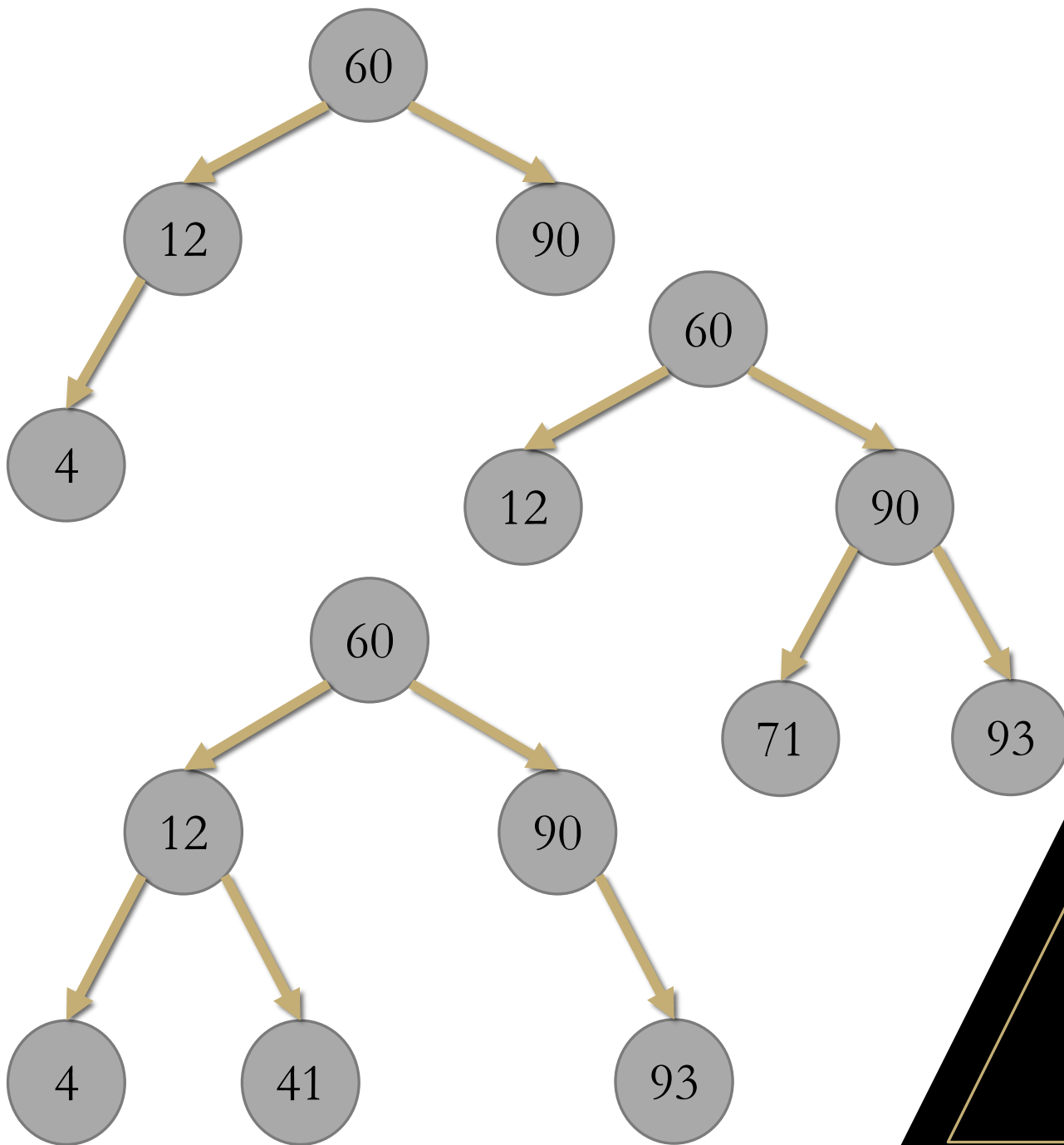


The minimum value
is in the leftmost
node, and the
maximum value is in
the rightmost node.

Building a BST

Traverse the tree in inorder fashion.
Do you get a sorted list?

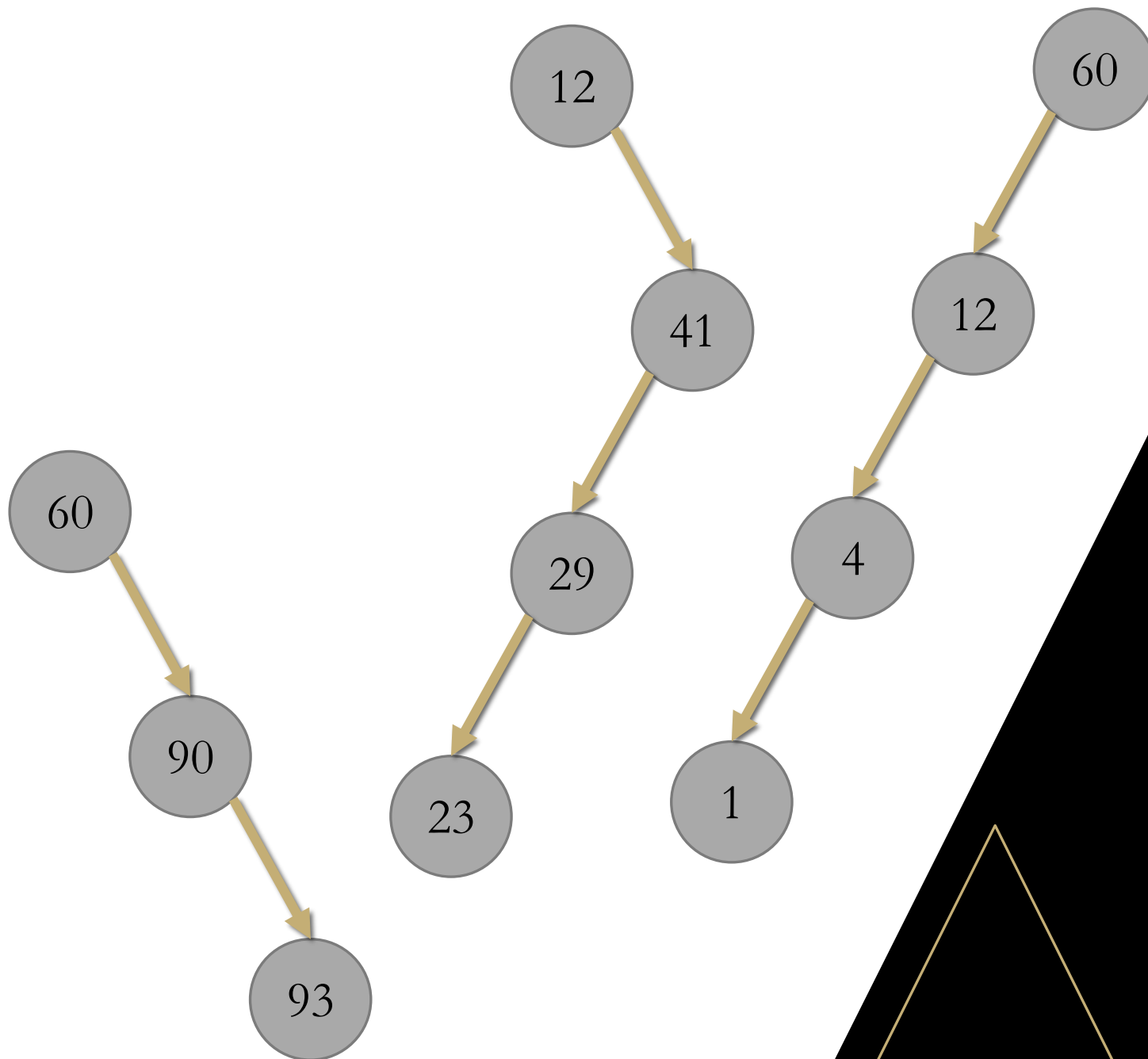




Balanced BST

- When the difference in the height of the left and right subtrees is 1 or 0 or -1, the BST is said to be balanced.
- A balanced BST is the kind of binary search tree that we mostly desire.





Worst BST

- The worst BST forms from a sorted sequence.
- Any BST of width 1 would be the worst BST.
- A tree of width 1 is actually a linked list.



Homework

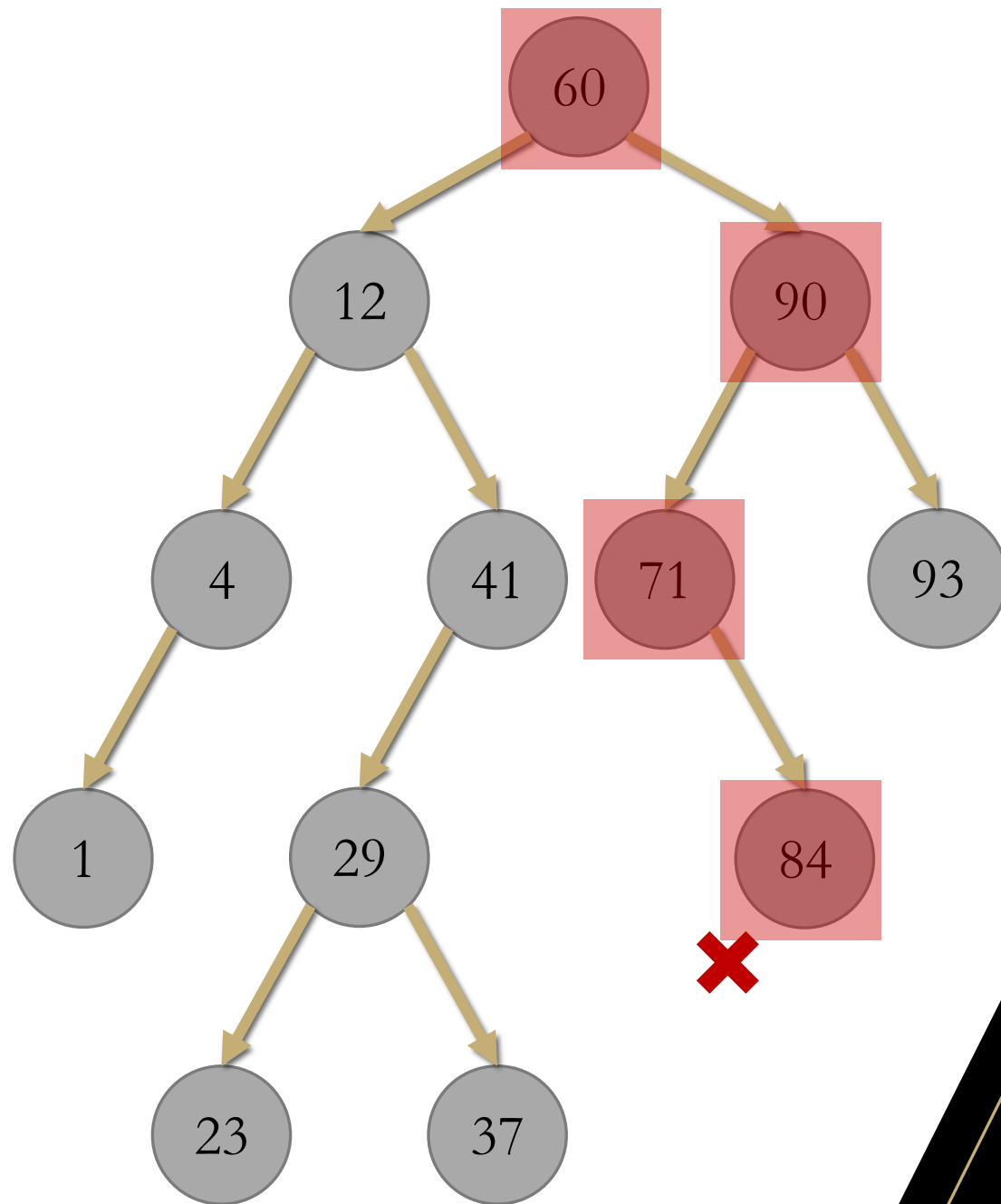
1. Build a BST for this sequence of numbers: 54, 16, 39, 46, 72, 80, 61, 64, 12, 45, 74.
2. Traverse the tree in inorder fashion.
3. Write code to find the minimum value in a BST. The function is called `minimum(root)`.
4. Write code to find the maximum value in a BST. The function is called `maximum(root)`.



```
def searchbst(node, target):
    if node is None: return [None, 0]
    if node.data == target: return [node, 1]
    if target < node.data:
        if node.lc is not None:
            if node.lc.data == target: return [node, 2]
            node = node.lc
        else: return [None, 0]
    else:
        if node.rc is not None:
            if node.rc.data == target: return [node, 3]
            node = node.rc
        else: return [None, 0]
    return searchbst(node, target)
```

Searching in a binary search tree

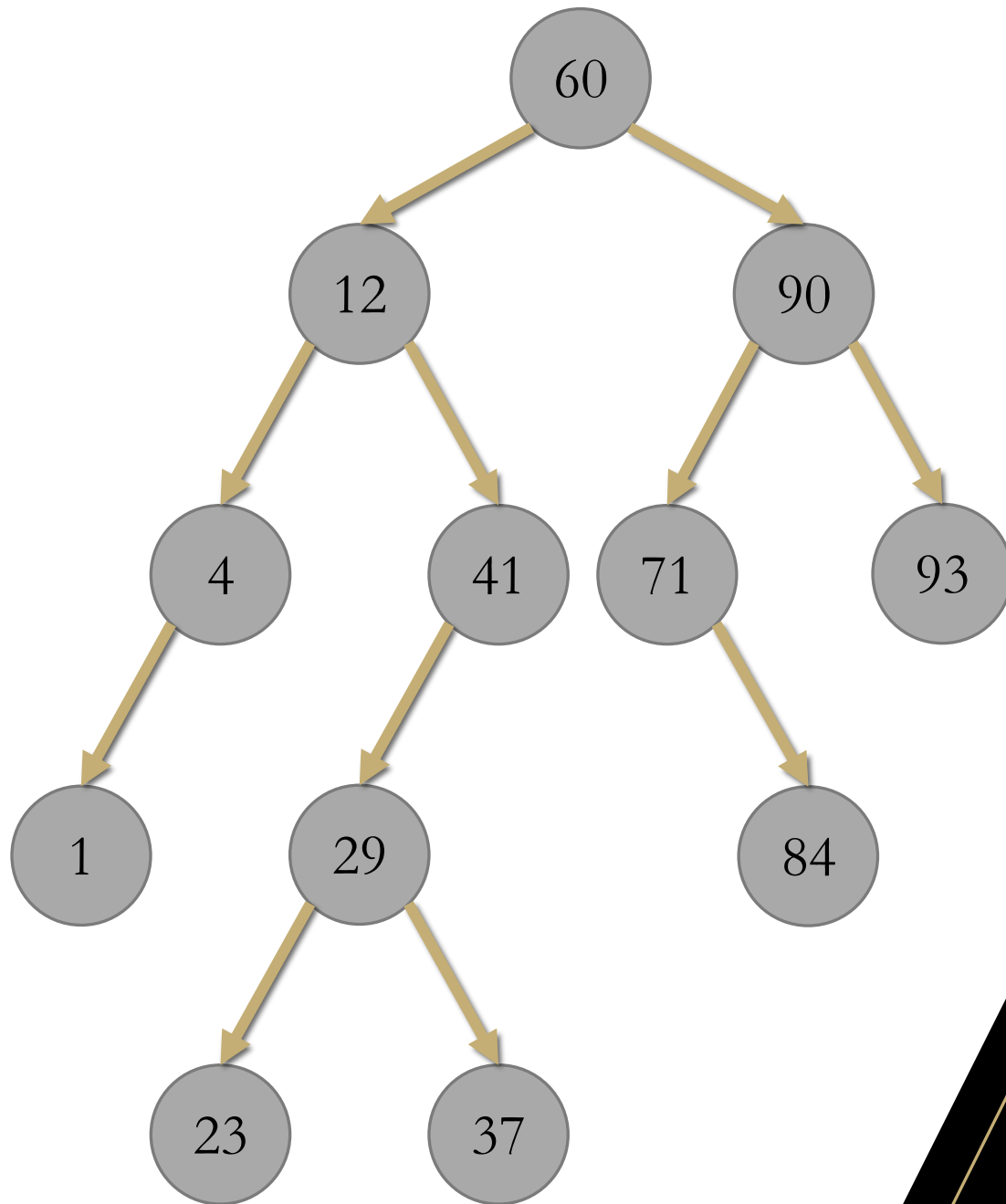




Searching in a
binary search
tree

Find 83.





Searching in a binary search tree

Find 29, 36.



Time: $O(h)$
Space: $O(h)$

```
def searchbst(node, target):  
    if node is None: return [None, 0]  
    if node.data == target: return [node, 1]  
    if node.data > target:  
        if node.lc is not None:  
            if node.lc.data == target: return [node, 2]  
            node = node.lc  
        else: return [None, 0]  
    else:  
        if node.rc is not None:  
            if node.rc.data == target: return [node, 3]  
            node = node.rc  
        else: return [None, 0]  
    return searchbst(node, target)
```

Searching in a binary search tree

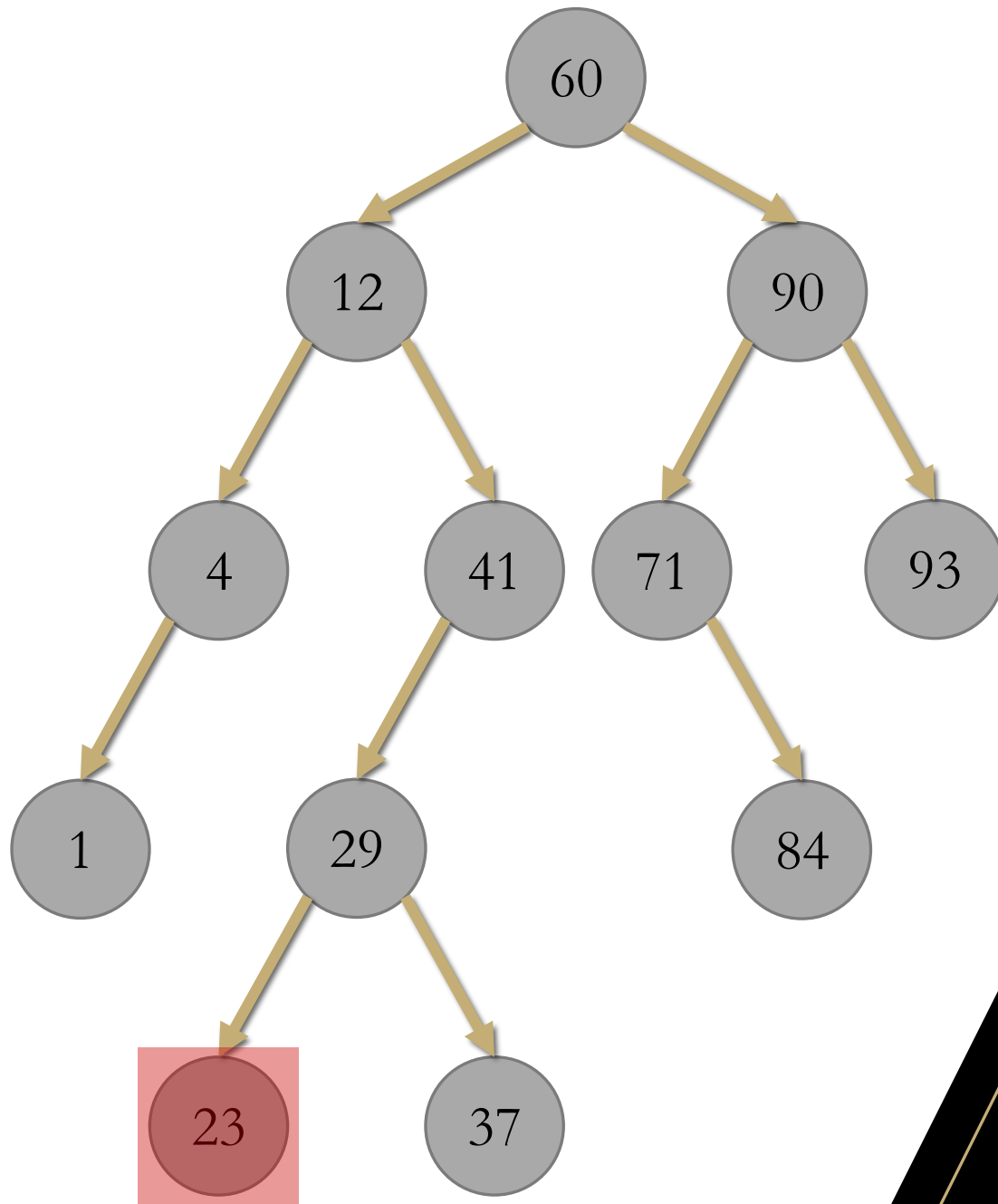
- In the worst case, we call as many recursive function instances as the height of the given tree.
- The number of activation records pushed onto the run-time stack is $h - 1$.



Deleting from a binary search tree

- If the node to be deleted, the **victim**, is a leaf, simply go the parent of the victim and set the relevant pointer to None. Deleted!
- If the victim has one child, make the parent of the victim adopt the victim's child. Delete the victim.
- If the victim has two children, find its logical successor. If the logical successor has a right child, it is adopted by the logical successor's parent, and the data value of the logical successor is copied to the victim's data field. The logical successor is deleted.
- The logical successor of a node is the minimum value in its right subtree.

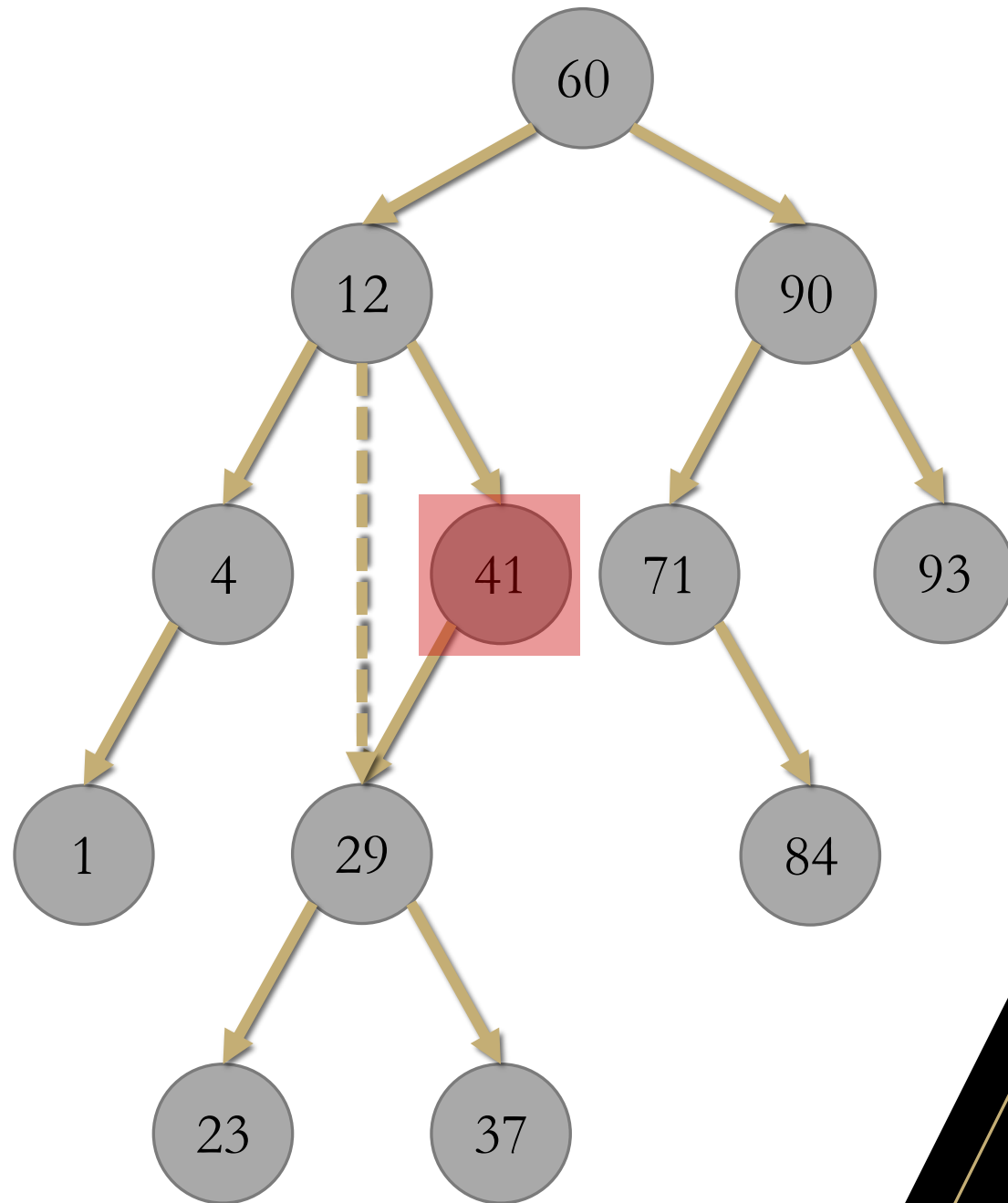




Deleting from a binary search tree

Delete 23.

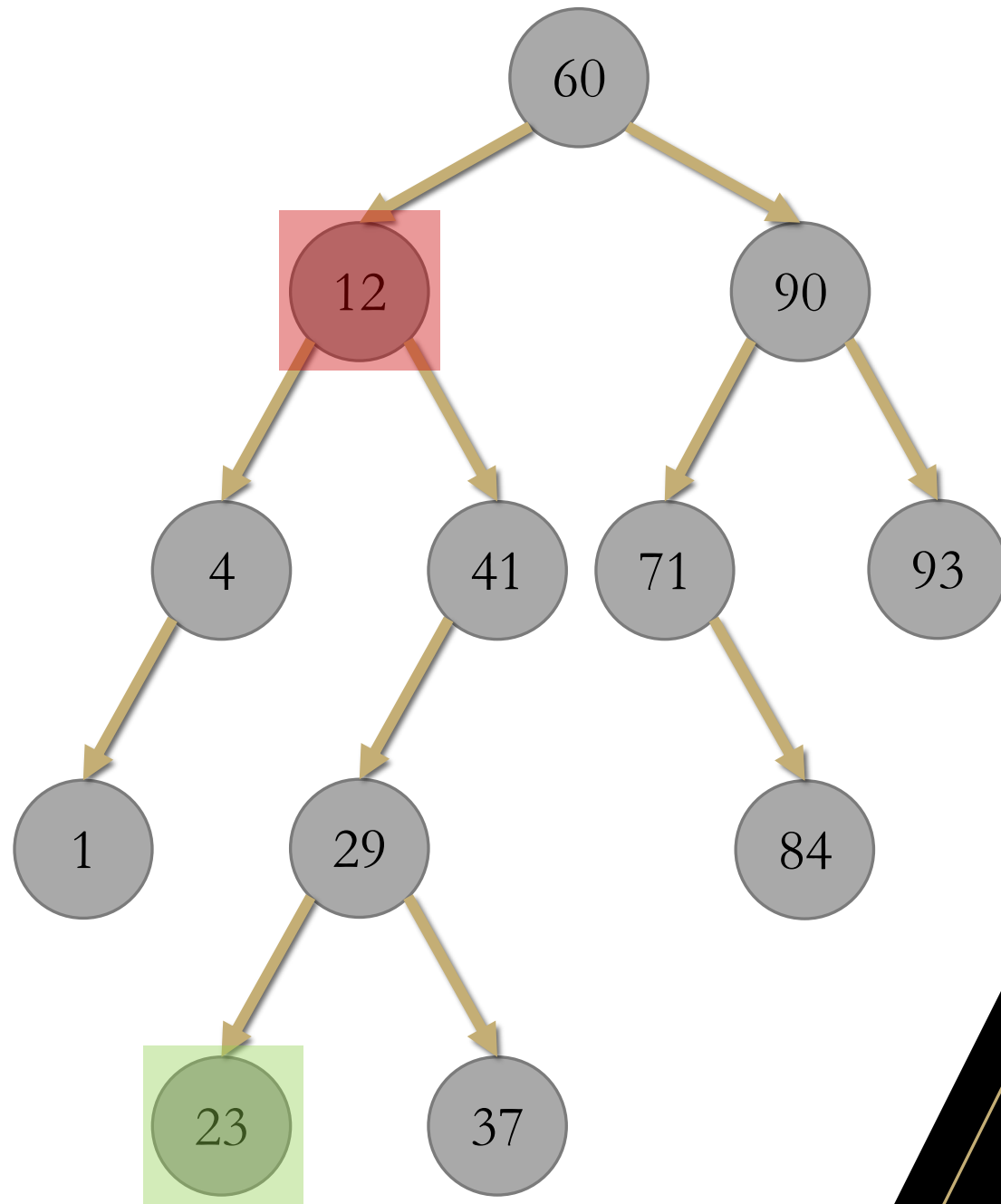




Deleting from a binary search tree

Delete 41.

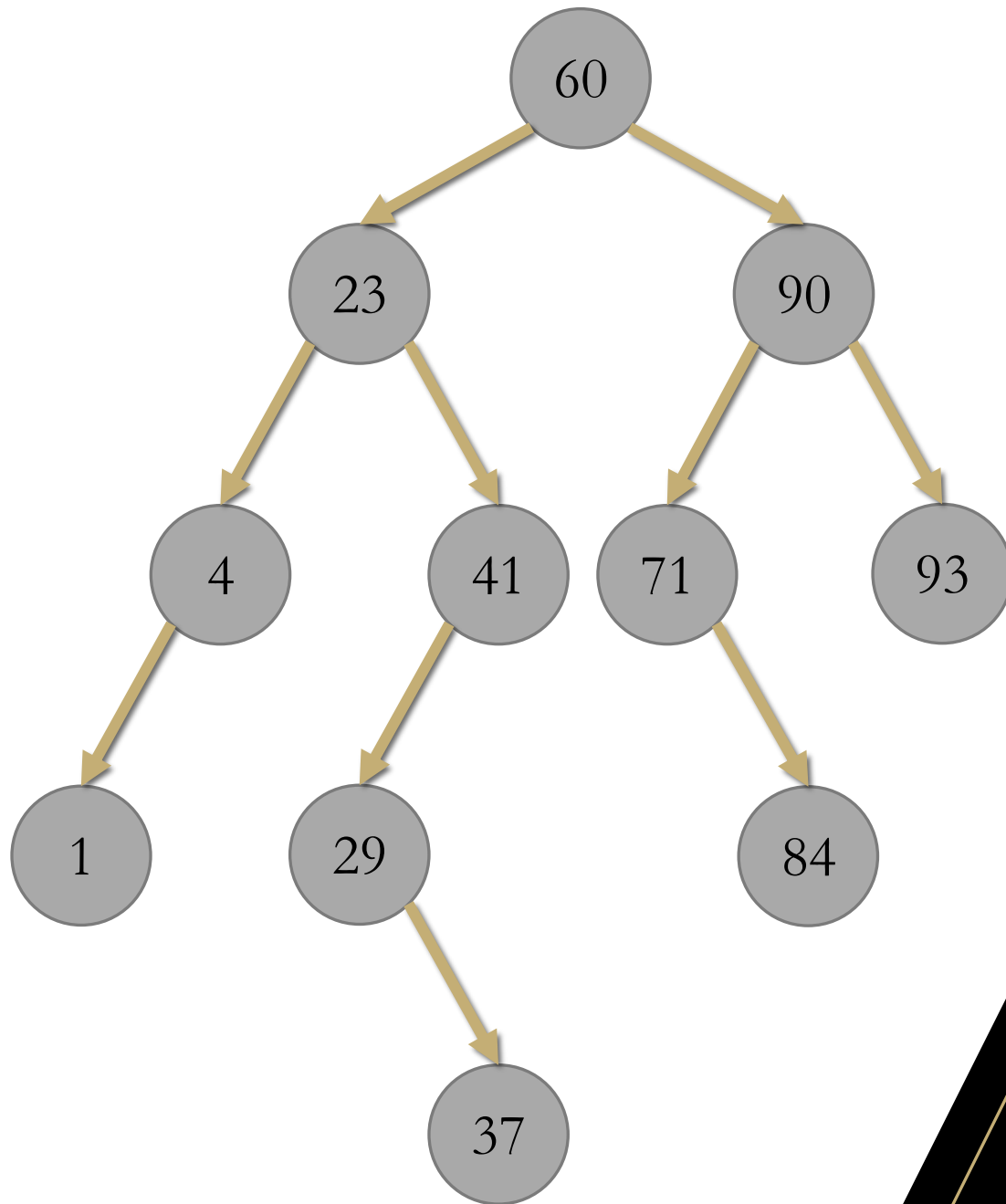




Deleting from a binary search tree

Delete 12.

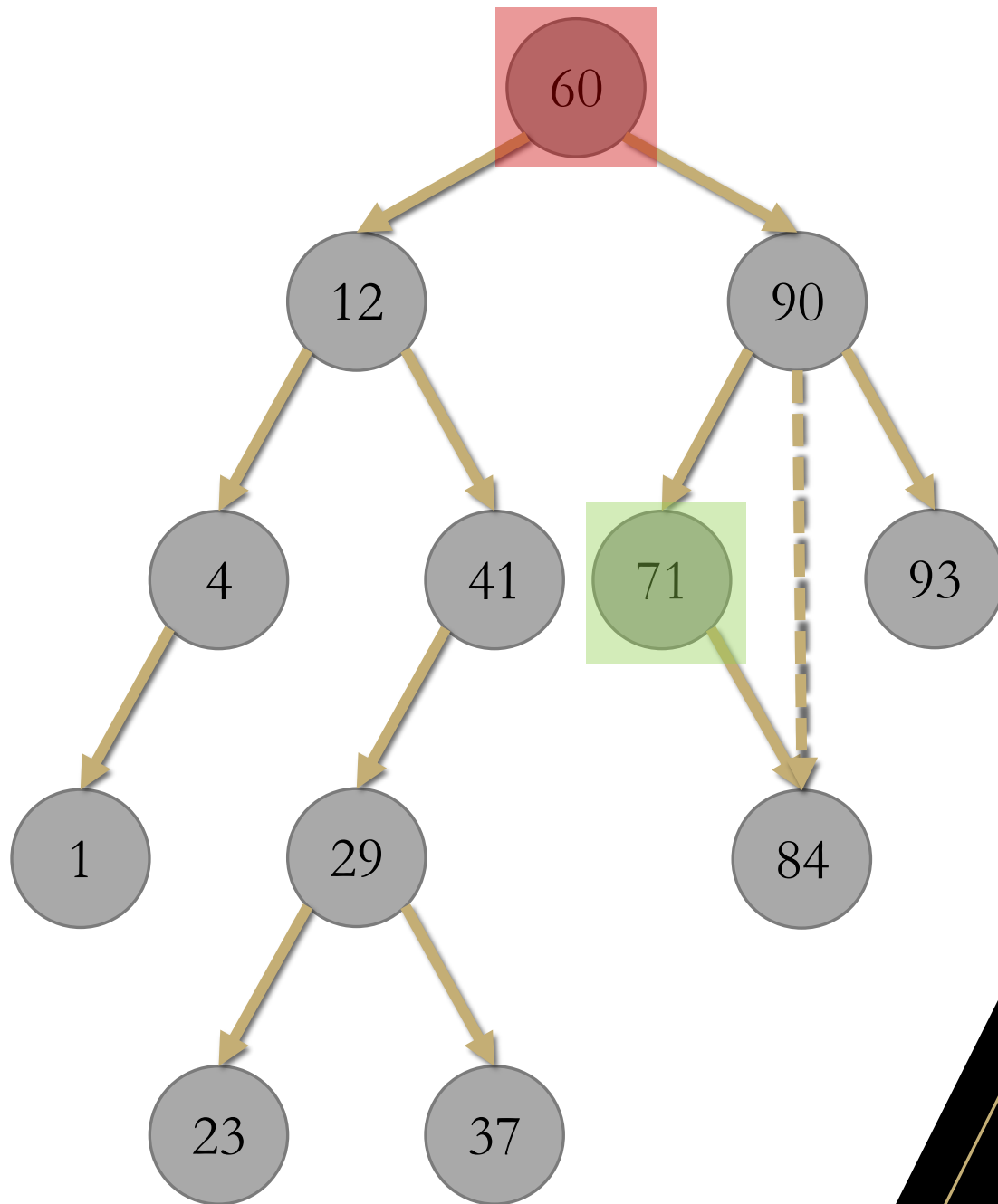




Deleting from a binary search tree

Delete 12.

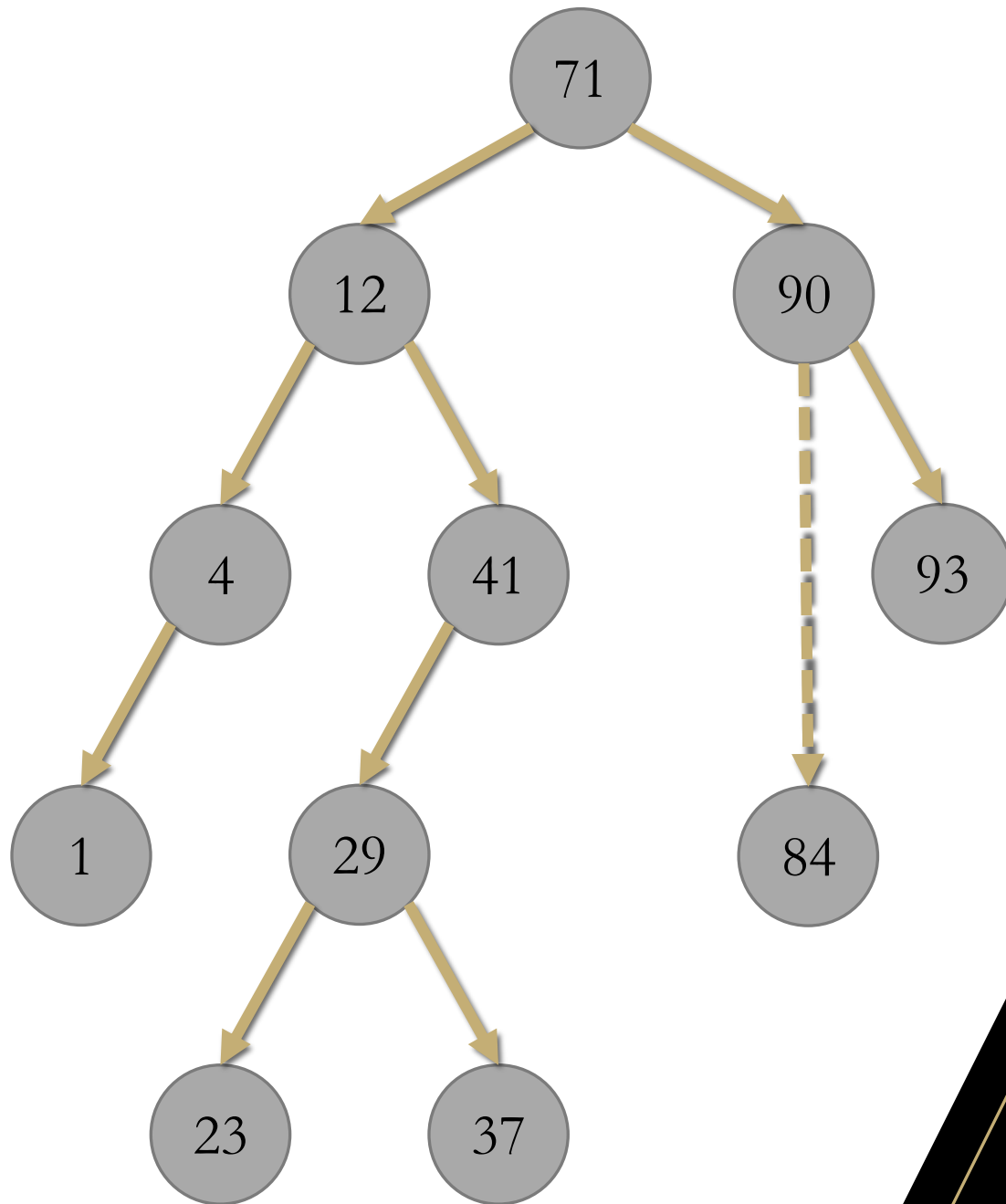




Deleting from a
binary search
tree

Delete 60.

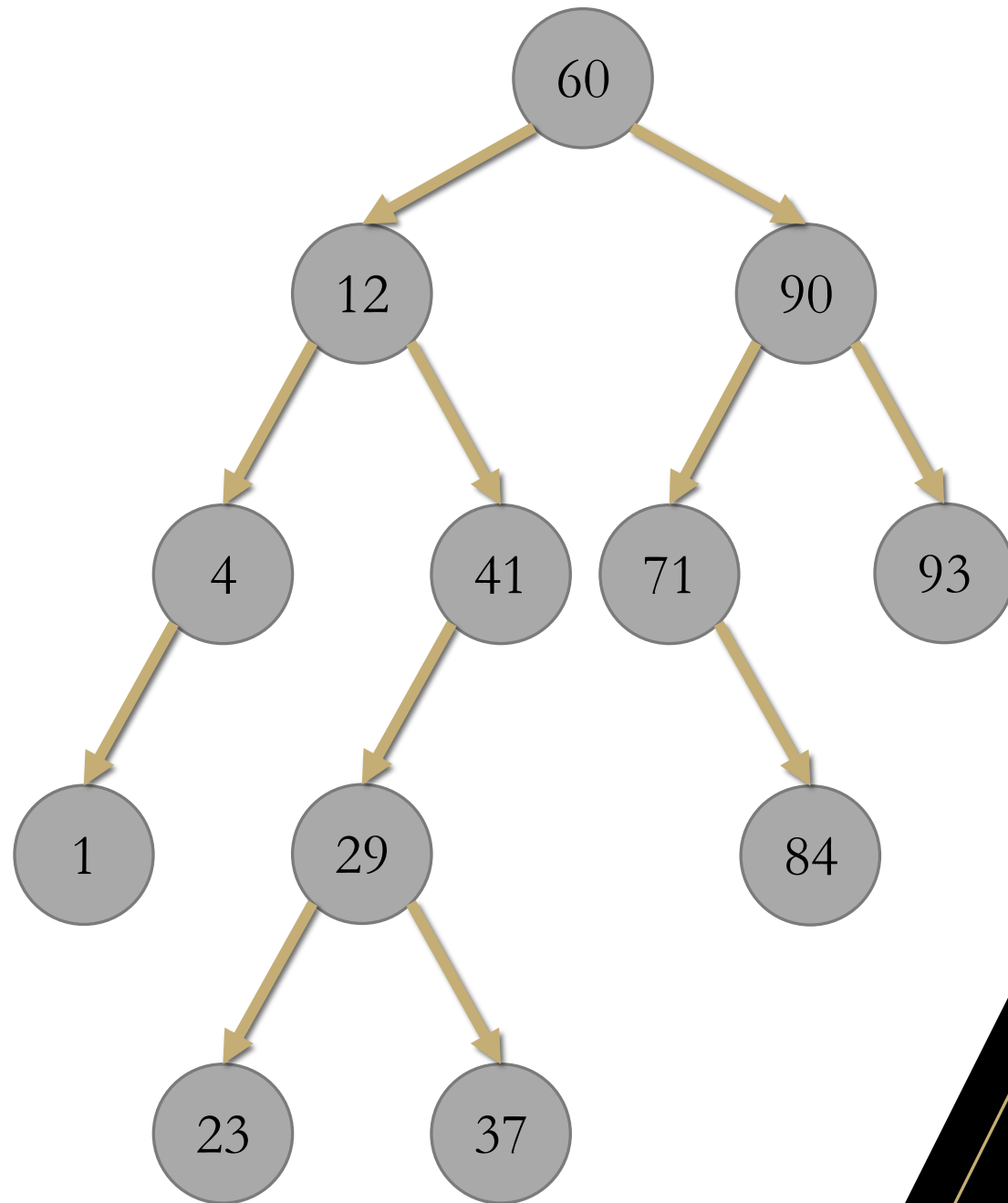




Deleting from a binary search tree

Delete 60.





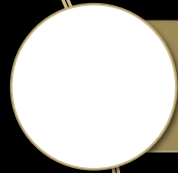
Deleting from a binary search tree

Delete 90.

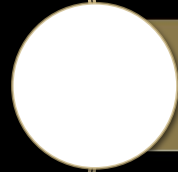




We used heaps to sort data.



We learned to build binary search trees.



We learned to search in a binary search tree.



We learned to delete in a binary search tree.



We examined all codes for time and space complexities.

So, what did we learn today?



Things to do

Read the book!

Email suggestions on content or quality of this lecture at uroojain@neduet.edu.pk

Note your questions and put them up in the relevant online session.

